LINCROS
by Sankhya

DOCUMENTAÇÃO TÉCNICA · APACHE NIFI

JsonToXmlTemplate

Monte o XML completo a partir de um JSON, com um template. Sem SplitJson, sem MergeContent e sem XML pela metade.

```
{"numero": 123, "itens": [{"sku": "A1"}, {"sku": "B7"}]}
↓ <Pedido numero="{{numero}}">{{#each itens}}<Item>{{sku}}</Item>{{/each}}</Pedido>
<Pedido numero="123"><Item>A1</Item><Item>B7</Item></Pedido>
```

Versão	Compatível com	Data
1.1.0	Apache NiFi 1.23 (Java 8+)	Novembro de 2026

EM RESUMO

O XML sai inteiro, em um passo só

Montar XML no NiFi com `SplitJson`, `ReplaceText` e `MergeContent` funciona, mas é frágil: um lote que fecha antes da hora gera um XML pela metade. O **JsonToXmlTemplate** recebe o JSON inteiro e monta o documento completo a partir de um template, escrito do jeito que o XML precisa ficar.

1
processor no lugar de Split + Replace + Merge

0
XML pela metade: sai completo ou vai para failure

12
filtros de formatação (data, número, CNPJ...)

36
testes automatizados, todos passando

Sumário

1 Por que este processor
2 Como funciona
3 Primeiro template em 5 minutos
4 Parte única
5 Listas (arrays)
6 Condições e contexto
7 Filtros
8 Configuração e saídas
9 Templates em arquivo
10 Migrando do MergeContent
11 Exemplos reais
12 Erros, limites e perguntas frequentes

A ideia em uma imagem

TEMPLATE (VOCÊ ESCREVE)

```
<Pedido numero="{{numero}}">
  <Cliente>{{cliente.nome}}</Cliente>
  {{#each itens}}
  <Item seq="{{@number}}">{{sku}}</Item>
  {{/each}}
</Pedido>
```

XML GERADO

```
<Pedido numero="123">
  <Cliente>Silva &amp;amp; Filhos</Cliente>
  <Item seq="1">A1</Item>
  <Item seq="2">B7</Item>
</Pedido>
```

A **parte única** funciona como um ReplaceText: cada `{{campo}}` é trocado pelo valor do JSON. O trecho entre `{{#each}}` e `{{/each}}` se repete para cada item da lista.

CAPÍTULO 1

Por que este processor

O NiFi não tem um jeito nativo de gerar um XML com layout específico (atributos, namespaces, cabeçalho e itens) a partir de um JSON. Por isso, muitos fluxos montam o XML "em pedaços".

O jeito comum hoje

JSON → SplitJson → ReplaceText (cada item) → MergeContent (cabeçalho + itens + rodapé) → XML

Funciona na maior parte do tempo, mas tem pontos frágeis:

Problema	O que acontece
Lote fecha antes da hora	Se uma parte atrasa, o *Max Bin Age* fecha o lote incompleto e o XML sai sem itens ou sem o fechamento.
Lotes misturados	Sem um *Correlation Attribute* bem definido, itens de pedidos diferentes caem no mesmo XML.
Ordem trocada	Com várias tarefas em paralelo, os itens podem chegar fora de ordem.
Cluster	Partes do mesmo documento em nós diferentes nunca se encontram.
Caracteres especiais	Um `&` ou `<` no dado, colado com ReplaceText, quebra o XML.
Difícil de manter	O layout fica espalhado em vários processors: cabeçalho num lugar, item em outro, rodapé em outro.

Com o JsonToXmlTemplate

JSON → JsonToXmlTemplate → XML

Tudo ou nada

O documento sai completo em `success` ou vai para `failure` com o motivo. Nunca sai pela metade.

Layout num lugar só

O template mostra o XML final inteiro: cabeçalho, itens e totais. Fácil de ler, revisar e versionar.

Escape automático

`&`, `<`, `>` e aspas nos dados viram entidades XML. O XML não quebra por causa do conteúdo.

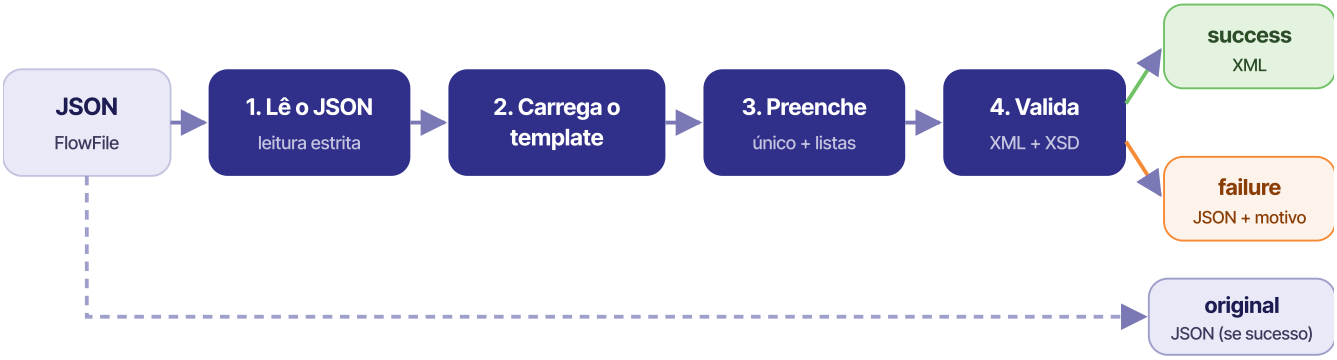
Validado antes de sair

Todo XML gerado é verificado. Opcionalmente, também é validado contra o XSD do parceiro.

CAPÍTULO 2

Como funciona

Cada FlowFile com JSON passa por cinco etapas. Se qualquer uma falhar, o JSON original vai para `failure` com a explicação.



1. **Lê o JSON.** Usa uma leitura estrita. Se o JSON estiver quebrado, a mensagem indica linha e coluna e sugere usar o **RepairXmlJson** antes. Números são guardados exatamente como vieram, sem arredondamento.
2. **Carrega o template.** Vem da propriedade ou de um arquivo, que pode ser escolhido por atributo. O template é compilado uma vez e reaproveitado; se o arquivo mudar, é relido automaticamente.
3. **Preenche.** Troca os `{{campos}}` da parte única, repete os blocos `{{#each}}` para cada item e aplica condições e filtros. Todo valor é escapado para XML.
4. **Valida.** O XML gerado precisa estar bem formado. Se configurado, tags vazias são removidas e o documento é validado contra o XSD.
5. **Entrega.** O XML vai para `success` com `mime.type=application/xml` e o nome do arquivo trocado para `.xml`. O JSON de entrada vai para `original`.

CAPÍTULO 3

Primeiro template em 5 minutos

1. **Instale o pacote.** Copie `nifi-repair-nar-1.1.0.nar` para `$NIFI_HOME/extensions/`. O NiFi carrega sozinho em alguns segundos, sem reiniciar. Em cluster, copie em todos os nós.
2. **Adicione o processor.** Arraste um Processor para o canvas e busque **JsonToXmlTemplate**.
3. **Cole o template** na propriedade **Template XML**. Comece pelo XML que você já gera hoje e troque os valores por `{{campos}}`.
4. **Conecte as saídas.** `success` segue o fluxo; `failure` vai para uma quarentena; `original` pode ser auto-terminada.
5. **Teste** com um JSON real e confira o XML na fila de `success`.

Exemplo completo

JSON DE ENTRADA

```
{"numero": 123, "emissao": "2026-11-02T10:15:00-03:00",
 "cliente": {"nome": "Silva & Filhos Ltda", "cnpj": "12.345.678/0001-99"},
 "itens": [{"sku": "A1", "descricao": "Tubo 3\" aço", "qtd": 2, "valor": 150},
           {"sku": "B7", "descricao": "Conexão T", "qtd": 10, "valor": 12.5}],
 "total": 425}
```

TEMPLATE

```
<Pedido numero="{{numero}}">
  <Emissao>{{emissao | date:'dd/MM/yyyy'}}
</Emissao>
  <Cliente cnpj="{{cliente.cnpj | digits}}">
    {{cliente.nome}}
  </Cliente>
  <Itens total="{{itens.length}}">
    {{#each itens}}
    <Item seq="{{@number}}">
      <Sku>{{sku}}</Sku>
      <Desc>{{descricao}}</Desc>
      <Valor>{{valor | number:2}}</Valor>
    </Item>
    {{/each}}
  </Itens>
  <Total>{{total | number:2}}</Total>
</Pedido>
```

XML GERADO

```
<Pedido numero="123">
  <Emissao>02/11/2026</Emissao>
  <Cliente cnpj="12345678000199">
    Silva &amp;amp; Filhos Ltda
  </Cliente>
  <Itens total="2">
    <Item seq="1">
      <Sku>A1</Sku>
      <Desc>Tubo 3&amp;quot; aço</Desc>
      <Valor>150.00</Valor>
    </Item>
    <Item seq="2">
      <Sku>B7</Sku>
      <Desc>Conexão T</Desc>
      <Valor>12.50</Valor>
    </Item>
  </Itens>
  <Total>425.00</Total>
</Pedido>
```

Repare

As linhas que só têm `{{#each itens}}` e `{{/each}}` não deixam linha em branco no XML. O `&` e as aspas dos dados foram escapados automaticamente.

CAPÍTULO 4

Parte única

É o cabeçalho, os totais e qualquer dado que aparece uma vez só. Funciona como um ReplaceText: você escreve o XML fixo e marca onde entra cada valor.

Como apontar para um campo

Marcação	Pega	Exemplo de resultado
`{{numero}}`	Campo na raiz do JSON	`123`
`{{cliente.nome}}`	Campo dentro de um objeto (quantos níveis quiser)	`Silva & Filhos`
`{{itens[0].sku}}`	Um item específico da lista (começa em 0)	`A1`
`{{itens.length}}`	Quantidade de itens da lista	`2`
`{{@attr.filename}}`	Um atributo do FlowFile	`pedido-123.json`
`{{{trecho}}}`	O valor **sem escape**, para inserir um pedaço de XML pronto	`<b>1</b>`
`{{! comentário }}`	Nada: é um comentário que não vai para o XML	—

Escape automático

Todo valor inserido com `{{ }}` é convertido para ser seguro dentro de um texto ou atributo XML:

No JSON	No XML	Observação
`&`	`&`	Ex.: *Silva & Filhos*
`<` e `>`	`<` e `>`	Ex.: *peso < 10kg*
`"` e `'`	`"` e `'`	Seguro dentro de atributos com qualquer tipo de aspas
Caracteres de controle	removidos	São proibidos em XML 1.0

> **Cuidado com {{{ }}}**
> As três chaves inserem o valor exatamente como está. Use só quando o campo já contém XML válido. Se o trecho estiver quebrado, o XML final é rejeitado e vai para `failure`.

Campo que não veio no JSON

Por padrão, um campo ausente vira texto vazio. Para outros comportamentos:

- **Valor padrão:** `{{obs | default:'Sem observação'}}` usa o texto quando o campo está ausente, nulo ou vazio.
- **Tag opcional:** `{{#if obs}}<Obs>{{obs}}</Obs>{{/if}}` só gera a tag se houver valor.
- **Modo estrito:** com *Campo ausente no JSON = Falhar*, qualquer campo faltando manda o FlowFile para `failure`, com o nome do campo e a linha do template na mensagem.
- **Limpeza no final:** com *Remover elementos vazios = true*, tags que ficaram sem valor e sem atributos são removidas do XML.

CAPÍTULO 5

Listas (arrays)

Escreva o trecho de **um** item entre `{{#each lista}}` e `{{/each}}`. Ele se repete para cada item, e dentro dele os campos são lidos do item atual.

Variáveis do laço

Marcação	Valor	Para 3 itens
`{{@number}}`	Posição começando em 1 (sequência)	1, 2, 3
`{{@index}}`	Posição começando em 0	0, 1, 2
`{{@count}}`	Total de itens da lista	3, 3, 3
`{{@first}}` · `{{@last}}`	É o primeiro / o último? (use em `{{#if}}`)	true no 1º / no 3º
`{{this}}`	O próprio item (para listas de textos)	`["a","b"]` → a, b

Usando a parte única dentro da lista

Se um campo não existe no item, ele é procurado no nível de cima. Assim, o número do pedido pode ir em cada item sem esforço:

```
{{#each itens}}<Item pedido="{{numero}}" sku="{{sku}}"/>{{/each}}
→ <Item pedido="123" sku="A1"/><Item pedido="123" sku="B7"/>
```

Se o item tiver um campo com o mesmo nome, use `{{../numero}}` para o nível de cima ou `{{@root.numero}}` para a raiz do JSON.

Lista dentro de lista

Blocos `{{#each}}` podem ser aninhados, por exemplo CT-e → notas → volumes:

```
<Cte>
  {{#each notas}}
  <Nota chave="{{chave}}">
    {{#each volumes}}
    <Volume seq="{{@number}}" peso="{{peso | number:3}}"/>
    {{/each}}
  </Nota>
  {{/each}}
</Cte>
```

Situações especiais

Situação	O que acontece
Lista vazia, nula ou ausente	Nenhuma repetição. Use `{{else}}` para gerar algo no lugar: `{{#each notas}}...{{else}}<SemNotas/>{{/each}}`
Veio um objeto onde se esperava lista	É tratado como lista de um item. Acontece muito quando o JSON foi convertido de XML e havia só um item.

CAPÍTULO 6

Condições e contexto

Condições

```
{{#if desconto}}<Desconto>{{desconto}}</Desconto>{{/if}}
{{#if status == 'A'}}<Ativo/>{{else}}<Inativo/>{{/if}}
{{#if peso > 1000}}<CargaPesada/>{{/if}}
{{#if entregue ≥ previsto}}<NoPrazo/>{{/if}}     compara dois campos
{{#unless obs}}<SemObservacao/>{{/unless}}   o contrário do if
```

Operadores: `==` `≠` `>` `<` `≥` `≤`. Quando os dois lados são números, a comparação é numérica (`10 > 9`); senão, é por texto. O lado direito pode ser texto entre aspas, número, `true` , `false` , `null` ou outro campo.

O que conta como "verdadeiro" num {{#if campo}}

Valor do campo	Resultado
Texto com conteúdo, qualquer número (inclusive `0`), `true` , lista ou objeto com itens	verdadeiro
Campo ausente, `null` , `false` , texto vazio `""` , lista vazia `[]` , objeto vazio `{}`	falso

O número 0 conta como verdadeiro de propósito: um desconto ou frete zerado normalmente precisa aparecer no XML. Para esconder zeros, use `{{#if desconto > 0}}` .

Entrar num objeto: {{#with}}

Evita repetir o caminho quando vários campos vêm do mesmo objeto:

```
{{#with transportadora}}
<Transportadora cnpj="{{cnpj | digits}}" ie="{{ie}}">{{nome}}</Transportadora>
{{/with}}
```

Se o objeto não existir, o bloco não é gerado (ou gera o `{{else}}` , se houver).

Navegação

Marcação	Significado
`{{campo}}`	Procura no item atual e, se não achar, nos níveis de cima
`{{../campo}}`	Campo do nível imediatamente acima (pode repetir: `../../campo`)
`{{@root.campo}}`	Campo a partir da raiz do JSON
`{{this}}` ou `{{.}}`	O valor atual inteiro

CAPÍTULO 7

Filtros

Filtros formatam o valor antes de entrar no XML. Use `|` depois do campo e encadeie quantos precisar: `{{nome | trim | upper}}`.

Filtro	Exemplo	Resultado
default	`{{obs | default:'N/A'}}`	`N/A` quando o campo está ausente, nulo ou vazio
upper / **lower**	`{{uf | upper}}`	`SP`
trim	`{{nome | trim}}`	Tira espaços das pontas
number	`{{valor | number:2}}`	`1520.75`. Arredonda meio para cima e aceita `1520.75`, `"1,5"` e `"1.234,56"`
date	`{{emissao | date:'dd/MM/yyyy'}}`	`02/11/2026`
date (com entrada)	`{{d | date:'yyyy-MM-dd':'ddMMyyyy'}}`	Formato de saída e, depois, o formato de entrada
digits	`{{cnpj | digits}}`	`12345678000199` (só os dígitos)
lpad	`{{numero | lpad:9}}`	`000004521` (completa à esquerda; padrão `0`)
rpad	`{{cod | rpad:10:'.'}}`	`AB........` (completa à direita; padrão espaço)
truncate	`{{obs | truncate:60}}`	Primeiros 60 caracteres
replace	`{{tel | replace:' ':''}}`	Troca um texto por outro
json	`{{extra | json}}`	O valor como texto JSON (escapado)

Datas

O filtro `date` reconhece sozinho estes formatos de entrada:

Formato no JSON	Exemplo
ISO com fuso	`2026-11-02T10:15:00-03:00`
ISO sem fuso / só data	`2026-11-02T10:15:00` · `2026-11-02`
Brasileiro	`02/11/2026 10:15:00` · `02/11/2026 10:15` · `02/11/2026`
Outros	`2026-11-02 10:15:00` · `20261102` · milissegundos (`1793618100000`)

O formato de saída segue o padrão Java (`dd` dia, `MM` mês, `yyyy` ano, `HH:mm:ss` hora, `XXX` fuso). Exemplo para o padrão da SEFAZ: `{{emissao | date:"yyyy-MM-dd'T'HH:mm:ssXXX"}}`. Datas sem fuso usam a propriedade **Fuso horário** (padrão `America/Sao_Paulo`).

> **Filtros com dois pontos no argumento**
>
> Coloque o argumento entre aspas simples ou duplas: `date:'HH:mm'`. Os dois pontos dentro das aspas não são confundidos com separador.

CAPÍTULO 8

Configuração e saídas

Propriedades

Propriedade	Padrão	Descrição
Template XML	—	O template escrito direto na propriedade. Erros de sintaxe aparecem na tela do NiFi, com a linha.
Arquivo de template	—	Caminho de um arquivo com o template. Aceita Expression Language. Preencha este **ou** o anterior, nunca os dois.
Modo de saída	Um XML por JSON	**Um XML por item do array**: se o JSON tiver uma lista na raiz, gera um XML para cada item. Substitui o SplitJson.
Campo ausente no JSON	Deixar vazio	**Falhar**: campo usado no template e ausente no JSON manda para failure (exceto com `default`).
Remover elementos vazios	false	Remove do XML final as tags sem texto, sem filhos e sem atributos.
Arquivo XSD	—	Opcional. Valida o XML contra o XSD antes de enviar. Aceita Expression Language.
Fuso horário	America/Sao_Paulo	Usado pelo filtro `date` quando a data não tem fuso.

Saídas

Saída	Conteúdo	Quando
success	O XML gerado (UTF-8)	Tudo deu certo
original	O JSON de entrada, sem alteração	Junto com o success. Pode ser auto-terminada
failure	O JSON de entrada + atributo `xml.template.error`	JSON inválido, erro no template, XML inválido ou fora do XSD

Atributos gravados no XML gerado

Atributo	Valor
`mime.type`	`application/xml`
`filename`	Nome original com `.xml` (`pedido-123.json` → `pedido-123.xml`). No modo por item: `lote-1.xml`, `lote-2.xml`...
`fragment.identifier` · `fragment.index` · `fragment.count`	Só no modo "um XML por item": identificam os XMLs que vieram do mesmo JSON

Os demais atributos do JSON de entrada são copiados para o XML gerado. Cada XML gerado registra um evento no Data Provenance.

Modo "um XML por item"

Para um JSON como `[{"numero": 1}, {"numero": 2}]`, o template é aplicado a cada item separadamente. No template, a raiz passa a ser o item, e `{{@number}}` e `{{@count}}` indicam a posição do item no lote.

CAPÍTULO 9

Templates em arquivo

Com os templates em arquivos, o layout fica fora do fluxo: dá para versionar no Git, revisar e reaproveitar em vários fluxos.

Um template por tipo de documento

A propriedade **Arquivo de template** aceita Expression Language. Um único processor atende vários layouts:

```
# escolhe pelo atributo "tipo" (pedido, cte, coleta...)
/opt/nifi/templates/${tipo}.xml

# escolhe pelo início do nome do arquivo: pedido-123.json → pedido.xml
/opt/nifi/templates/${filename:substringBefore('-')}.xml

# um template por parceiro
/opt/nifi/templates/${parceiro}/${tipo}.xml
```

Atualização sem parar o fluxo

O arquivo é lido uma vez e guardado em memória. Quando ele é alterado no disco, a nova versão é carregada automaticamente no próximo FlowFile. Não é preciso parar o processor.

Organização sugerida

```
/opt/nifi/templates/
├── pedido.xml
├── cte.xml
├── coleta.xml
└── xsd/
    ├── pedido.xsd
    └── cte.xsd
```

Combine com o **Arquivo XSD** usando o mesmo atributo: `/opt/nifi/templates/xsd/${tipo}.xsd`.

> **Em cluster**
> Os arquivos de template e XSD precisam existir no mesmo caminho em **todos os nós**. Uma pasta compartilhada (NFS) ou o deploy por Git em cada nó resolve.

> **Template na propriedade ou em arquivo?**
> Na propriedade é mais rápido para começar e o NiFi aponta erros de sintaxe na hora de configurar. Em arquivo é melhor para templates grandes, vários layouts e controle de versão; erros de sintaxe aparecem em `xml.template.error`.

CAPÍTULO 10

Migrando do MergeContent

A migração é juntar num template só o que hoje está espalhado entre ReplaceText e MergeContent.

Onde cada parte vai parar

Hoje	No template
Header do MergeContent	O início do template, com `{{campos}}` no lugar dos valores fixos ou de `${atributos}`
ReplaceText de cada item (depois do SplitJson)	O trecho dentro de `{{#each lista}} ... {{/each}}`
Demarcator do MergeContent	O texto entre os itens já fica dentro do `each` (quebra de linha, por exemplo)
Footer do MergeContent	O final do template, depois do `{{/each}}`
EvaluateJsonPath para extrair campos do cabeçalho	Não precisa: use `{{campo}}` direto
UpdateAttribute para formatar data/número	Filtros: `date`, `number`, `lpad`...

Antes e depois

JSON → EvaluateJsonPath → SplitJson → EvaluateJsonPath → ReplaceText → MergeContent → XML

JSON → JsonToXmlTemplate → XML

Passo a passo

1. **Pegue um XML real** que o fluxo atual gera corretamente. Ele é o ponto de partida do template.
2. **Marque a parte única:** troque cada valor do cabeçalho pelo `{{campo}}` correspondente no JSON.
3. **Marque a lista:** apague os itens repetidos, deixe só um e coloque `{{#each lista}}` antes e `{{/each}}` depois.
4. **Aplique os filtros** para datas, valores e documentos, conforme o layout exige.
5. **Rode em paralelo:** mande o mesmo JSON para o fluxo antigo e para o novo e compare os XMLs.
6. **Desligue o fluxo antigo** quando os resultados baterem. Se o parceiro tiver XSD, configure-o para validar cada documento.

CAPÍTULO 11

Exemplos reais

Resultados do teste de ponta a ponta em um **Apache NiFi 1.23.2** real. O template foi escolhido pelo nome do arquivo (`${filename:substringBefore('-')}.xml`). Os XMLs gerados foram conferidos por um leitor de XML independente.

CT-e com notas e volumes (lista dentro de lista)

success

JSON

```
{"numero": 4521, "transportadora": "TransLog", "notas": [{"chave": "3526...2345", "volumes": [{"peso": 10.5}, {"peso": 3}]}, {"chave": "3526...2346", "volumes": {"peso": 7.25}}]}
```

XML

```
<Conhecimento numero="000004521"><Transportadora>TransLog</Transportadora>
<Nota chave="3526...2345"><Volume seq="1" peso="10.500"/><Volume seq="2" peso="3.000"/></Nota>
<Nota chave="3526...2346"><Volume seq="1" peso="7.250"/></Nota></Conhecimento>
```

Na segunda nota, "volumes" veio como objeto e não como lista; foi tratado como lista de um item.

CT-e sem notas

success

JSON

```
{"numero": 999, "transportadora": "Vazio", "notas": []}
```

XML

```
<Conhecimento numero="000000999"><Transportadora>Vazio</Transportadora><SemNotas/></Conhecimento>
```

O bloco {{else}} do {{#each notas}} gerou <SemNotas/>.

Pedido com caracteres especiais, data, CNPJ e valores

success

JSON (TRECHO)

```
"cliente": {"nome": "Silva & Filhos Ltda", "cnpj": "12.345.678/0001-99"}, "observacao": "Entregar no galpão 2 <fundos>", "itens": [{"descricao": "Tubo 3\" aço inox", "valor": 150}, ...]
```

XML (TRECHO)

```
<Cliente cnpj="12345678000199">SILVA &amp;amp; FILHOS LTDA</Cliente>
<Obs>Entregar no galpão 2 &lt;fundos&gt;</Obs>
<Descricao>Tubo 3&quot; aço inox</Descricao><Valor>150.00</Valor>
```

JSON quebrado

failure

JSON

```
{'numero': 124, 'itens': [
```

XML.TEMPLATE.ERROR

```
JSON inválido: esperava aspas duplas no nome do campo (linha 1, coluna 2). Dica: use o RepairXmlJson antes deste processor.
```

> **Combinação recomendada**
>
> Se os JSONs podem chegar com defeitos (aspas simples, vírgulas sobrando), use os dois processors do pacote em sequência: **RepairXmlJson** → **JsonToXmlTemplate**.

CAPÍTULO 12

Erros, limites e perguntas frequentes

Mensagens de erro mais comuns

Mensagem	O que fazer
`'{{/if}}' na linha 4 fecha o bloco '#each' aberto na linha 2`	Os blocos estão cruzados. Feche primeiro o bloco aberto por último.
`Bloco '#each' aberto na linha 2 não foi fechado`	Falta o `{{/each}}` correspondente.
`Filtro 'maiusculo' desconhecido na linha 3`	Confira o nome do filtro; a mensagem lista os disponíveis.
`Campo 'cliente.cnpj' não existe no JSON (linha 5 do template)`	No modo estrito: corrija o caminho ou use `| default:''` se o campo for opcional.
`JSON inválido: ... (linha 1, coluna 2)`	Coloque o RepairXmlJson antes, ou corrija a origem.
`O XML gerado não é válido (linha 3, coluna 10)`	Uma tag do template não está fechada, ou um `{{{ }}}` inseriu XML quebrado.
`XML não segue o XSD (linha 7): ...`	O dado não atende ao layout do parceiro (campo obrigatório, tamanho, tipo). Veja a linha indicada.
`Arquivo de template não encontrado`	Confira o caminho e o atributo usado na Expression Language, em todos os nós.

Limites

- **Memória:** o JSON e o XML são montados inteiros em memória; indicado para documentos de até algumas dezenas de MB. A saída é sempre UTF-8.
- **DOCTYPE:** por segurança, XMLs com declaração `<!DOCTYPE>` são rejeitados na validação.
- **Expression Language** vale nos caminhos de arquivo (template e XSD), não dentro do texto do template. Para atributos dentro do template, use `{{@attr.nome}}`.

Perguntas frequentes

Preciso saber programar para escrever um template?

Não. O template é o próprio XML de destino com marcações. Quem já monta XML com ReplaceText e MergeContent consegue escrever um template. Para gerar dois layouts do mesmo JSON, use dois processors em paralelo, cada um com seu template.

Consigo gerar o XML a partir de um CSV ou de um banco?

Sim: converta para JSON antes (por exemplo, `ConvertRecord` com `JsonRecordSetWriter`, ou `ExecuteSQLRecord` com saída JSON) e use o template normalmente.

Funciona no NiFi 2.x? Onde está o código?

Foi feito e testado para o NiFi 1.23; para o 2.x precisaria ser recompilado. O código está no repositório `jaapa07/Claude`, branch `feature/nifi-repair-xml-json`, pasta `nifi-repair-bundle/`. O pacote `nifi-repair-nar-1.1.0.nar` traz este processor e o RepairXmlJson.



Logística não precisa ser complicada para ser eficiente.

JsonToXmlTemplate 1.1.0 · Documentação técnica · Novembro de 2026